In [ ]:
```python
print("Hello world")
```

In [ ]:
```python
#variable-containers to store data
a=5
b=6
a+b
```

In [ ]:
```python
#Data type
a=10
b="hey"
c=5.5
d=[1,2,3]
e=(1,2,3)
f={a:'1'}
print(type(a))
print(type(b))
print(type(c))
print(type(d))
print(type(e))
print(type(f))
```

In [ ]:
```python
# typr casting-1)implicit Typecasting
a=5
b=6.5
print(a+b)# float ,done manually by compiler
#2) explicit type casting
a=10
b=9.5
print(int(a+b))# done by programmer manually,int
```

In [ ]:
```python
#string & methods and slicing--
name = "Yugam"
print(name)
print(name.upper())
print(name.endswith("am"))
print(name.endswith("jerry"))
print(name.lower())
print(name.count("a"))
print(name.count("r"))
s = "CodeWithMe"
print(len(s))
print(s[0:5])
print(s[1:-1])
print(s[::-1])
print(s[1:12])
print(s[-12:-1])
```

In [ ]:
```python
#  🔹 OPERATORS IN PYTHON (Brief Theory)
# Operators are used to perform operations on variables and values.

# 1. Arithmetic Operators (+, -, *, /, //, %, **)
a = 10
b = 5
```

```python
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Floor Division:", a // b)
print("Modulus:", a % b)
print("Exponent:", a ** b)

# 2. Comparison Operators (==, !=, >, <, >=, <=)
print("Equal:", a == b)
print("Not Equal:", a != b)
print("Greater:", a > b)
print("Less:", a < b)
print("Greater Equal:", a >= b)
print("Less Equal:", a <= b)

# 3. Logical Operators (and, or, not)
x = True
y = False

print("AND:", x and y)
print("OR:", x or y)
print("NOT x:", not x)

# 4. Bitwise Operators (&, |)
a = 5
b = 3

print("Bitwise AND:", a & b)
print("Bitwise OR:", a | b)

# 5. Assignment Operators
a = 10
a += 5
print("a += 5:", a)

a -= 5
print("a -= 5:", a)

a *= 2
print("a *= 2:", a)

# 6. Membership & Identity Operators
lst = [1, 2, 3]
x = 2

print("x in list:", x in lst)
print("x not in list:", x not in lst)

a = [1,2]
b = [1,2]

print("a is b:", a is b)
print("a is not b:", a is not b)
```

In [ ]:
```
a=input()
b=input("Enter the val of b:")
print(type(b))
c=int(input("Enter val of c:"))
print(type(c))
```

In [ ]:
```
'''Operator Precedence
Python follows PEMDAS (Parentheses, Exponents, Multiplication/Division, Addition/Su

Parentheses () – Highest precedence, operations inside parentheses are evaluated fi
Exponents ** – Power calculations (e.g., 2 ** 3 → 8).
Multiplication *, Division /, Floor Division //, Modulus % – Evaluated from left to
Addition +, Subtraction - – Evaluated from left to right.'''
result = 10 + 2 * 3  # Multiplication happens first: 10 + (2 * 3) = 16
print(result)

result = (10 + 2) * 3  # Parentheses first: (10 + 2) * 3 = 36
print(result)

result = 2 ** 3 ** 2  # Right-to-left exponentiation: 2 ** (3 ** 2) = 2 ** 9 = 512
print(result)
```

In [ ]:
```
#Basic if Statement:The if statement allows you to execute a block of code only if
x = 10
if x > 5:
    print("x is greater than 5")
#if-else Statement:The else block executes when the if condition is False.
x = 3
if x > 5:
    print("x is greater than 5")
else:
    print("x is not greater than 5")
#if-elif-else Statement:When multiple conditions need to be checked sequentially, u
x = 5
if x > 10:
    print("x is greater than 10")
elif x > 5:
    print("x is greater than 5 but not more than 10")
elif x == 5:
    print("x is exactly 5")
else:
    print("x is less than 5")
#Using if-else in Data Science:Conditional statements are widely used in data scien
#Example: Categorizing Data
age = 25
if age < 18:
    category = "Minor"
elif age < 65:
    category = "Adult"
else:
    category = "Senior Citizen"

print("Category:", category)
#Example: Applying Conditions on Pandas DataFrame
'''import pandas as pd
```

```
data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Score': [85, 40, 75]}
df = pd.DataFrame(data)

df['Result'] = df['Score'].apply(lambda x: 'Pass' if x >= 50 else 'Fail')
print(df)'''
```

In [ ]:

```
#The match-case statement, introduced in Python 3.10, provides pattern matching sim
def http_status(code):
    match code:
        case 200:
            return "OK"
        case 400:
            return "Bad Request"
        case 404:
            return "Not Found"
        case 500:
            return "Internal Server Error"
        case _:
            return "Unknown Status"

print(http_status(200))  # Output: OK
print(http_status(404))  # Output: Not Found

#Example: Matching Data Structures
point = (3, 4)

match point:
    case (0, 0):
        print("Origin")
    case (x, 0):
        print(f"X-Axis at {x}")
    case (0, y):
        print(f"Y-Axis at {y}")
    case (x, y):
        print(f"Point at ({x}, {y})")
```

In [ ]:

```
#String Formatting and f-strings
name = "Alice"
age = 25
print("My name is {} and I am {} years old.".format(name, age))

#. Using f-Strings
name = "Alice"
age = 25
print(f"My name is {name} and I am {age} years old.")

#Formatting Numbers:-
pi = 3.14159
print(f"Pi rounded to 2 decimal places: {pi:.2f}")

#Padding and Alignment
print(f"{'Python':<10}")  # Left-align
print(f"{'Python':>10}")  # Right-align
print(f"{'Python':^10}")  # Center-align
```

In [ ]:
```python
#Loops in Python
#1.for loop:Used to iterate over sequences like lists, tuples, and strings.
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)
# range function:
for i in range(10):
    print(i)
print("----------------------------------------------------------------------")

#2. While Loop:Runs as long as a condition is True.
count = 0
while count < 3:
    print(count)
    count += 1
print("----------------------------------------------------------------------")


#3. Loop Control Statements
#break → Exits the loop.
#continue → Skips to the next iteration.
#pass → Does nothing (used as a placeholder).
for i in range(5):
    if i == 3:
        break  # Stops the loop at 3
    print(i)
print("----------------------------------------------------------------------")
```

List and List Methods A list in Python is an ordered, mutable collection of elements. It can contain elements of different types.

Creating a List:

In [ ]:
```python
# Empty list
my_list = []

# List with elements
numbers = [1, 2, 3, 4, 5]

# Mixed data types
mixed_list = [1, "Hello", 3.14, True]
```

Common List Methods Method Description Example append(x) Adds an element x to the end of the list. my_list.append(10) extend(iterable) Extends the list by appending all elements from an iterable. my_list.extend([6, 7, 8]) insert(index, x) Inserts x at the specified index. my_list.insert(2, "Python") remove(x) Removes the first occurrence of x in the list. my_list.remove(3) pop([index]) Removes and returns the element at index (last element if

index is not provided). my_list.pop(2) index(x) Returns the index of the first occurrence of x. my_list.index(4) count(x) Returns the number of times x appears in the list. my_list.count(2) sort() Sorts the list in ascending order. my_list.sort() reverse() Reverses the order of the list. my_list.reverse() copy() Returns a shallow copy of the list. new_list = my_list.copy() clear() Removes all elements from the list. my_list.clear()

In [ ]:
```
fruits = ["apple", "banana", "cherry"]
fruits.append("orange")
print(fruits)  # ['apple', 'banana', 'cherry', 'orange']

fruits.sort()
print(fruits)  # ['apple', 'banana', 'cherry', 'orange']
```

Tuples and Tuple Methods A tuple in Python is an ordered, immutable collection of elements. It is similar to a list, but once created, its elements cannot be modified.

Creating a Tuple:

In [ ]:
```
# Empty tuple
empty_tuple = ()

# Tuple with elements
numbers = (1, 2, 3, 4, 5)

# Mixed data types
mixed_tuple = (1, "Hello", 3.14, True)

# Single element tuple (comma is necessary)
single_element = (42,)
```

Common Tuple Methods Method Description Example count(x) Returns the number of times x appears in the tuple. my_tuple.count(2) index(x) Returns the index of the first occurrence of x. my_tuple.index(3)

Tuple Characteristics Immutable: Once created, elements cannot be changed.

Faster than lists: Accessing elements in a tuple is faster than in a list.

Can be used as dictionary keys: Since tuples are immutable, they can be used as keys in dictionaries.

Accessing Tuple Elements

In [ ]:
```
my_tuple = (10, 20, 30, 40)

# Indexing
print(my_tuple[1])  # 20

# Slicing
print(my_tuple[1:3])  # (20, 30)
```

In [ ]:
```
# Packing
person = ("Alice", 25, "Engineer")

# Unpacking
name, age, profession = person
print(name)  # Alice
print(age)   # 25
```

When to Use Tuples?

When you want an unchangeable collection of elements.

When you need a faster alternative to lists.

When storing heterogeneous data (e.g., database records, coordinates)

- What is a Set?

A set in Python is an unordered, mutable collection of unique elements. It automatically removes duplicate values.

In [ ]:
```
# Creating sets
empty_set = set()
numbers = {1, 2, 3, 4, 5}
mixed_set = {1, "Hello", 3.14, True}

# From list
unique_numbers = set([1, 2, 2, 3, 4, 4, 5])
print("Unique Numbers:", unique_numbers)

# Methods
my_set = {1, 2, 3}

# add
my_set.add(4)
print("After add:", my_set)

# update
my_set.update([5, 6])
print("After update:", my_set)

# remove
my_set.remove(2)
print("After remove:", my_set)

# discard
my_set.discard(10)  # no error
print("After discard:", my_set)

# pop
removed = my_set.pop()
print("Popped element:", removed)

# copy
```

```python
new_set = my_set.copy()
print("Copied set:", new_set)

# clear
temp_set = {1, 2, 3}
temp_set.clear()
print("After clear:", temp_set)

# Set operations
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}

print("Union:", set1 | set2)
print("Intersection:", set1 & set2)
print("Difference:", set1 - set2)
print("Symmetric Difference:", set1 ^ set2)

# Methods form
print("Union method:", set1.union(set2))
print("Intersection method:", set1.intersection(set2))

# Subset & Superset
A = {1, 2}
B = {1, 2, 3}

print("A subset of B:", A.issubset(B))
print("B superset of A:", B.issuperset(A))
```

A dictionary in Python is an unordered, mutable, and key-value pair collection. It allows efficient data retrieval and modification. Dictionaries in Python are ordered as of Python 3.7

Creating a Dictionary:

In [ ]:
```python
# Empty dictionary
empty_dict = {}

# Dictionary with key-value pairs
student = {
    "name": "Alice",
    "age": 25,
    "grade": "A"
}

# Using dict() constructor
person = dict(name="John", age=30, city="New York")
```

In [ ]:
```python
# Using keys
print(student["name"])  # Alice

# Using get() (avoids KeyError if key doesn't exist)
print(student.get("age"))  # 25
print(student.get("height", "Not Found"))  # Default value
```

Method	Description	Example
`keys()`	Returns all keys in the dictionary.	`student.keys()`
`values()`	Returns all values in the dictionary.	`student.values()`
`items()`	Returns key-value pairs as tuples.	`student.items()`
`get(key, default)`	Returns value for key, or default if key not found.	`student.get("age", 0)`
`update(dict2)`	Merges another dictionary into the dictionary.	`student.update({"age": 26})`
`pop(key, default)`	Removes key and returns its value (or default if key not found).	`student.pop("grade")`
`popitem()`	Removes and returns the last inserted key-value pair.	`student.popitem()`
`setdefault(key, default)`	Returns value for key, else sets it to default.	`student.setdefault("city", "Unknown")`
`clear()`	Removes all items from the dictionary.	`student.clear()`
`copy()`	Returns a shallow copy of the dictionary.	`new_dict = student.copy()`

In [ ]:
```
student = {"name": "Alice", "age": 25, "grade": "A"}

# Adding a new key-value pair
student["city"] = "New York"

# Updating an existing value
student["age"] = 26

# Removing an item
student.pop("grade")

# Iterating over a dictionary
for key, value in student.items():
    print(key, ":", value)

# Output:
# name : Alice
# age : 26
# city : New York
```

In [ ]:
```
# Creating a dictionary using comprehension
squares = {x: x**2 for x in range(1, 6)}
print(squares)  # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

- Key Properties of Dictionaries:

Unordered (Insertion Order Maintained) Dictionaries do not support indexing like lists. Python 3.6+ se insertion order maintain hota hai. Keys must be Unique and Immutable Keys unique hone chahiye.

Immutable types use hote hain (string, number, tuple). Values can be of any type

Values mutable ya immutable dono ho sakte hain. Example: list, string, integer, etc.

1. Introduction to Lists 1)Create a list fruits = ["apple", "banana", "cherry"]. Print the first fruit. Replace "banana" with "orange". Print the length of the list. 2)Create a list of numbers from 1 to 10. Print the first three numbers using slicing. Print the last three numbers using slicing.

In [ ]:
```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])
fruits[1]="orange"
print(fruits)
print(len(fruits))

numlst=[1,2,3,4,5]
print(numlst[:3])
print(numlst[-3:])
```

2. List Methods 1)Start with numbers = [5, 2, 9, 1, 7] and do the following: Sort the list in ascending order. Append the number 10 to the list. Remove the number 2 from the list. 2)Create a list names = ["Alice", "Bob", "Charlie"] and use the insert() method to add "David" at index 1

In [ ]:
```
numbers = [5, 2, 9, 1, 7]
numbers.sort()
numbers.append(10)
numbers.remove(2)
print(numbers)

names = ["Alice", "Bob", "Charlie"]
names.insert(1,"David")
print(names)
```

3. Tuples and Operations on Tuples Create a tuple coordinates = (10, 20) and print both elements. Try to modify the tuple by setting coordinates[0] = 50 — note what happens. Convert the tuple to a list, change its first element to 50, and convert it back to a tuple.

In [ ]:
```
coordinates=(10,20)
for i in coordinates:
    print(i)
coordinates[0] = 50#tuple is immutable
coordinates=list(coordinates)
coordinates[0] = 50
coordinates=tuple(coordinates)
print(coordinates)
print(type(coordinates))
```

4. Sets and Set Methods 1) Create a set my_set = {1, 2, 3, 3, 4} and print it. (What happens to duplicate 3?) 2)Add 5 to the set, remove 2, and check if 4 is in the set. 3)Create two sets: a = {1, 2, 3} b = {3, 4, 5} Find their: Union Intersection Difference (a - b)

In [ ]:
```
my_set = {1, 2, 3, 3, 4}
print(my_set)#3 is printed only once as set removes duplicate item
my_set.add(5)
```

```
my_set.remove(2)
print(my_set)
print(4 in my_set)

a = {1, 2, 3}
b = {3, 4, 5}
print(a | b)
print( a & b)
print(a - b)
```

5. Dictionaries and Dictionary Methods 1)Create a dictionary student = {"name": "John", "age": 20, "grade": "A"} and: Print the value of "name". Change "grade" to "A+". Add a new key "city" with value "Delhi". 2)Create a dictionary of three friends and their phone numbers. Use: keys() to get all names values() to get all numbers items() to loop over key-value pairs and print them

In [ ]:
```
student = {"name": "John", "age": 20, "grade": "A"}
print(student["name"])
student["grade"]="A+"
student["city"]="Delhi"
print(student)

frnd={"yash":8796,"mayank":7999,"deep":5689}
print(frnd.keys())
print(frnd.values())
for x,y in frnd.items():
    print(f'Name:{x}\nnum:{y}')
    print("")
```

6. Bonus Challenges 1)Write a program that takes a list of numbers and removes all duplicates using a set. 2)Given a dictionary of products and their prices, find the product with the highest price. 3)Write a program that merges two dictionaries into one.

In [2]:
```
n = int(input("Kitne elements: "))
nums = []

for i in range(n):
    x = int(input("Enter number: "))
    nums.append(x)

print(nums)
nums=list(set(nums))
print(nums)
```

```
[5, 6, 4, 5]
[4, 5, 6]
```

In [7]:
```
item={"apple":500,"kiwi":54,"chiku":546}
a=max(item.values())
for x,y in item.items():
    if y==a:
        print("Highest price product is:")
        print(f'{x}={y}')
```

```
Highest price product is:
chiku=546
```

In [10]:
```
def merge_dict(a,b):
    result=a|b
```

```
    return result
a={'a':5,'b':15}
b={'c':89,'d':45}
print(merge_dict(a,b))
```

```
{'a': 5, 'b': 15, 'c': 89, 'd': 45}
```